Activity 5.1: Clustering

Objective:

This activity aims to perform classification of unlabeled data using K Means and Agglomerative Clustering

Intended Learning Outcomes (ILOs):

- Build the model using K Means and Agglomerative clustering.
- Determine the optimum number of clusters using elbow method.
- Visualize the dendogram produced by the agglomerative clustering

Resources:

- Jupyter Notebook
- Wholesale customers data.csv

Procedure:

We will used K-Means clustering to cluster the clients of a wholesale distributor based on their annual spending on diverse product categories, like milk, grocery, region, etc.

Import the needed libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.cluster import KMeans
```

Load the dataset and check the first five rows

```
df=pd.read_csv("wholesale_customers_data.csv")
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 440,\n  \"fields\": [\
n    {\n      \"column\": \"Channel\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n
\"max\": 2,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          1,\n          2\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Region\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0,\n        \"min\": 1,\n
\"max\": 3,\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          3,\n          1\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Fresh\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 12647,\n        \"min\": 3,\n
```

```
\"max\": 112151,\n        \"num_unique_values\": 433,\n
\"samples\": [\n          21117,\n          20398\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Milk\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 7380,\n        \"min\": 55,\n
\"max\": 73498,\n        \"num_unique_values\": 421,\n
\"samples\": [\n          8384,\n          7184\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Grocery\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 9503,\n
\"min\": 3,\n        \"max\": 92780,\n        \"num_unique_values\":
430,\n        \"samples\": [\n          5160,\n          3\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Frozen\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 4854,\n        \"min\": 25,\n        \"max\": 60869,\n
\"num_unique_values\": 426,\n        \"samples\": [\n          269,\n
7530\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Detergents_Paper\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 4767,\n        \"min\": 3,\n
\"max\": 40827,\n        \"num_unique_values\": 417,\n
\"samples\": [\n          302,\n          6740\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Delicassen\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
2820,\n        \"min\": 3,\n        \"max\": 47943,\n
\"num_unique_values\": 403,\n        \"samples\": [\n          14472,\
n          172\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


Check the descriptive statistics of the dataset

```
df.describe()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"Channel\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 155.12037873074365,\n
\"min\": 0.46805156947911486,\n        \"max\": 440.0,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
1.3227272727272728,\n          2.0,\n          0.46805156947911486\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Region\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
154.79240441475915,\n        \"min\": 0.7742724492300992,\n
\"max\": 440.0,\n        \"num_unique_values\": 6,\n
\"samples\": [\n          440.0,\n          2.543181818181818,\n
3.0\n        ],\n        \"semantic_type\": \"\",\n
```

```
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Fresh\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 37443.96728921031,\n        \"min\": 3.0,\n        \"max\":
112151.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
12000.297727272728,\n          8504.0,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Milk\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 24840.629356725192,\n
\"min\": 55.0,\n        \"max\": 73498.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
5796.265909090909,\n          3627.0,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Grocery\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
31275.067055578267,\n        \"min\": 3.0,\n        \"max\": 92780.0,\
n        \"num_unique_values\": 8,\n        \"samples\": [\n
7951.277272727273,\n          4755.5,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Frozen\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
20870.561028469416,\n        \"min\": 25.0,\n        \"max\":
60869.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
3071.931818181818,\n          1526.0,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Detergents_Paper\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
13891.619642837632,\n        \"min\": 3.0,\n        \"max\": 40827.0,\
n        \"num_unique_values\": 8,\n        \"samples\": [\n
2881.4931818181817,\n          816.5,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Delicassen\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
16572.2008225548,\n        \"min\": 3.0,\n        \"max\": 47943.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
1524.8704545454545,\n          965.5,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```


Since K-Means is a distance-based algorithm, the difference between the magnitudes of the data will create a problem. Let us apply featuring scaling to standardize the data.

```
# Use standard scaler to standardize the data
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
data_scaled = scaler.fit_transform(df)

# statistics of scaled data
pd.DataFrame(data_scaled).describe()
```

```
{"summary":"{\n  \"name\": \"pd\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": 0,\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 155.47401297080322,\n        \"min\": -
0.6902970861260346,\n        \"max\": 440.0,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
1.6148698540002276e-17,\n          1.4486516314475937,\n
1.001138304295671\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": 1,\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
155.56267540153488,\n        \"min\": -1.9953421178949025,\n
\"max\": 440.0,\n        \"num_unique_values\": 6,\n
\"samples\": [\n          440.0,\n          3.552713678800501e-16,\n
0.5906682852678581\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": 2,\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
155.21675147532295,\n        \"min\": -0.9496830857287227,\n
\"max\": 440.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          -3.431598439750484e-17,\n          -
0.27676022465581374,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": 3,\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 155.15771580825376,\n
\"min\": -0.7787950508196164,\n        \"max\": 440.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n          0.0,\n
-0.2942580226490891,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": 4,\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 155.17045097070678,\n
\"min\": -0.8373343668922592,\n        \"max\": 440.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n          -
4.0371746350005693e-17,\n          -0.3366684436852283,\n
440.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": 5,\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
155.03542557237495,\n        \"min\": -0.6283430304733164,\n
\"max\": 440.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          3.633457171500512e-17,\n          -
0.31880447003935,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": 6,\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 155.2060284846412,\n
\"min\": -0.6044164758928096,\n        \"max\": 440.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
2.4223047810003414e-17,\n          -0.4336004370553557,\n
440.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": 7,\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
154.84084293103095,\n        \"min\": -0.5402643872458722,\n
\"max\": 440.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          -8.074349270001138e-18,\n          -
```

```
0.1985766495208734,\n          440.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```

Build the K-Means model

```
# Create cluster of 2
kmeans = KMeans(n_clusters=2, init='k-means++', n_init=10,
random_state=42)

# fit the k means algorithm on scaled data
kmeans.fit(data_scaled)
```

```
KMeans(n_clusters=2, n_init=10, random_state=42)
```

Calculate the inertia to evaluate how well the formed clusters are.

```
# inertia on the fitted data
kmeans.inertia_
```

```
2599.387384912305
```

Interpret the result of the inertia

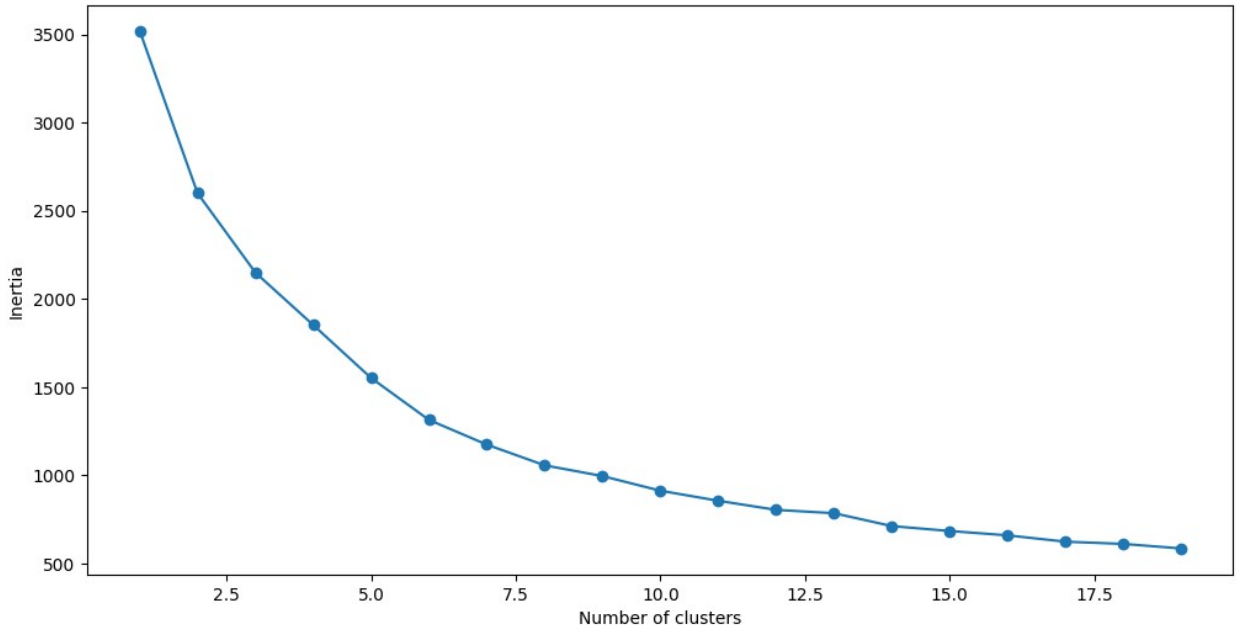
The inertia indicates how close and compact the clusters made by the data points when plotted in a graph. Higher inertia indicates poor clustering, which the current dataset shows.

Use the elbow curve to determine the optimum number of clusters in Python.

```
# fitting multiple k-means algorithms and storing the values in an
empty list
SSE = []
for cluster in range(1,20):
    kmeans = KMeans(n_clusters=cluster, init='k-means++', n_init=10,
random_state=42)
    kmeans.fit(data_scaled)
    SSE.append(kmeans.inertia_)

# converting the results into a dataframe and plotting them
frame = pd.DataFrame({'Cluster':range(1,20), 'SSE':SSE})
plt.figure(figsize=(12,6))
plt.plot(frame['Cluster'], frame['SSE'], marker='o')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia')
```

```
Text(0, 0.5, 'Inertia')
```



Interpret the result of the elbow curve.

The elbow curve indicates how the number of clusters used in K Means affect the inertia of the clusters, and it indicated that higher clusters will return lower and better inertia.

Build a model using 5 clusters and k-means ++ initialization

```
# k means using 5 clusters and k-means++ initialization
kmeans = KMeans(n_clusters=5, init='k-means++')
kmeans.fit(data_scaled)
pred = kmeans.predict(data_scaled)

frame = pd.DataFrame(data_scaled)
frame['cluster'] = pred
frame['cluster'].value_counts()
```

```
cluster
0    209
1    126
3     92
2     12
4      1
Name: count, dtype: int64
```

There are 208 data points belonging to cluster 1 (index 1), then 125 points in cluster 2 (index 2), and so on.

Compute the centroids of the K means cluster

```
centroids = kmeans.cluster_centers_
print(centroids)
```

```
[[-6.90297086e-01  5.90668285e-01  1.02719458e-01 -3.56479221e-01
  -4.43587243e-01  7.52790095e-02 -4.43491947e-01 -9.31373298e-02]
 [ 1.44865163e+00  1.69928497e-01 -3.06362835e-01  4.17500207e-01
   6.51879518e-01 -3.57269701e-01  6.76847966e-01  6.33174701e-03]
 [ 1.27040590e+00  5.19161179e-02  1.01697969e+00  3.88384518e+00
   3.77602602e+00  2.67775570e-01  3.91009990e+00  1.07099298e+00]
 [-5.74049873e-01 -1.58776439e+00  3.22040971e-02 -3.24809935e-01
  -3.91593636e-01  2.08356115e-01 -4.23471344e-01 -1.15896320e-01]
 [-6.90297086e-01  5.90668285e-01  1.96681731e+00  5.17550306e+00
   1.28721685e+00  6.90059988e+00 -5.54861977e-01  1.64784475e+01]]
```

What is the purpose of the centroids?

Centroids serves as central reference point for K Means clustering, which constantly changes as the means change.

Fit an agglomerative clustering model with two clusters.

```
from sklearn.cluster import AgglomerativeClustering

ag = AgglomerativeClustering(n_clusters=2, linkage='ward',
compute_full_tree=True)
ag = ag.fit(data_scaled)
df['agglom'] = ag.fit_predict(data_scaled)
```

Visualize the dendrogram produced by agglomerative clustering

```
from scipy.cluster import hierarchy
from matplotlib import colors
import seaborn as sns

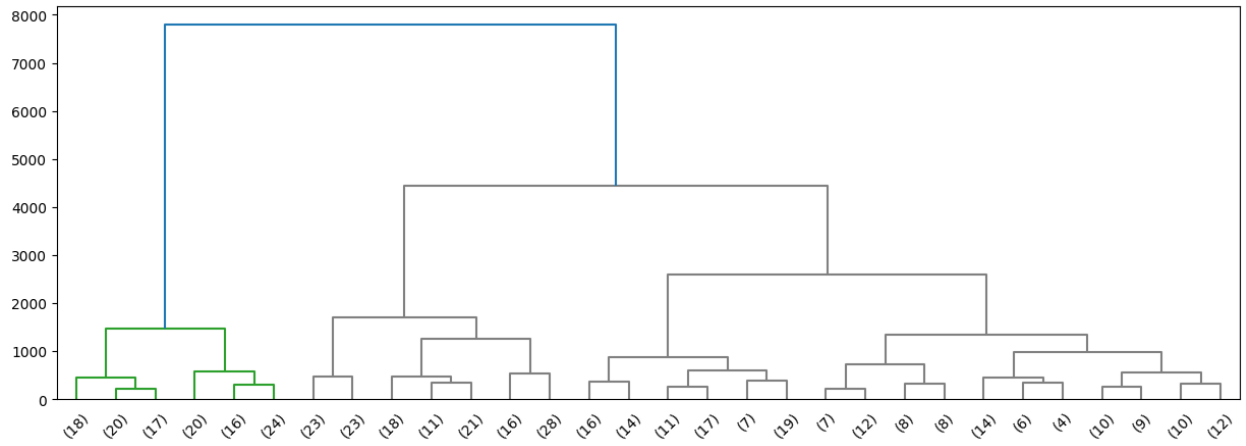
Z = hierarchy.linkage(ag.children_, method='ward')

fig, ax = plt.subplots(figsize=(15,5))

# Some color customization
dark_palette = sns.color_palette()
red = colors.to_hex(dark_palette[2])
blue = colors.to_hex(dark_palette[0])

hierarchy.set_link_color_palette([red, 'gray'])

den = hierarchy.dendrogram(Z, orientation='top',
                           p=30, truncate_mode='lastp',
                           show_leaf_counts=True, ax=ax,
                           above_threshold_color=blue)
```



Interpret the result of the dendogram

The dendogram indicates how the agglomorate clustering decides on the different clusters in the data points. The highlighted tree is the best among the others since it predicted the clusters efficiently.

Supplementary Activity:

- Choose your own dataset.
- Import the dataset
- Determine the number of datapoints, columns and data types
- Remove unneccesary columns
- Do data cleaning such as removing empty values(NaN), replacing missing data .
- Standardize the variables.
- Perform descriptive statistics such as mean, median and mode
- Perform data visualization
- Use k-means clustering to identify the groups of the dataset.
- Use 1 to 10 clusters
- Determine the optimum number of clusters
- Use agglomerative clustering and visualize the dendogram
- Compare the results obtained by the K means and agglomerative clustering.

#Dataset: (What Makes a Song Go Viral)

Import dataset

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("vedauv/what-makes-a-song-go-viral-10000-tracks")

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'what-makes-a-song-go-viral-10000-tracks' dataset.
Path to dataset files: /kaggle/input/what-makes-a-song-go-viral-10000-tracks
```

```
df_path = "/kaggle/input/what-makes-a-song-go-viral-10000-tracks/viral_songs.csv"

df = pd.read_csv(df_path)
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 10000,\n  \"fields\": [\n    {\n      \"column\": \"track_id\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10000,\n        \"samples\": [\n          \"track_06253\",\n          \"track_04685\",\n          \"track_01732\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"track_name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10000,\n        \"samples\": [\n          \"Track_06253\",\n          \"Track_04685\",\n          \"Track_01732\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"artist\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 1987,\n        \"samples\": [\n          \"Artist_0970\",\n          \"Artist_1802\",\n          \"Artist_1678\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"genre\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 10,\n        \"samples\": [\n          \"R&B\",\n          \"Electronic\",\n          \"Latin\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"release_year\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 4,\n        \"min\": 2010,\n        \"max\": 2025,\n        \"num_unique_values\": 16,\n        \"samples\": [\n          2013,\n          2018,\n          2016\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"duration_sec\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 49,\n        \"min\": 60,\n        \"max\": 433,\n        \"num_unique_values\": 299,\n        \"samples\": [\n          92,\n          303,\n          112\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"danceability\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.19261221213990126,\n        \"min\": 0.0,\n        \"max\": 1.0,\n        \"num_unique_values\": 916,\n        \"samples\": [\n          0.173,\n          0.059,\n          0.517\n
```

```
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"energy\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.1876859935743174,\n        \"min\": 0.005,\n        \"max\": 1.0,\n
\"num_unique_values\": 873,\n        \"samples\": [\n          0.122,\
n          0.797,\n          0.357\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"valence\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.1779835378507925,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 873,\n        \"samples\": [\n          0.985,\
n          0.504,\n          0.452\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"tempo_bpm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
19.01720569714834,\n        \"min\": 50.0,\n        \"max\": 177.8,\n
\"num_unique_values\": 974,\n        \"samples\": [\n          114.0,\
n          104.4,\n          122.7\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"acousticness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.2635737017177802,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 980,\n        \"samples\": [\n          0.283,\
n          0.836,\n          0.302\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"instrumentalness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.16122102897345286,\n        \"min\": 0.0,\n        \"max\": 0.934,\n
\"num_unique_values\": 669,\n        \"samples\": [\n          0.365,\
n          0.024,\n          0.584\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"speechiness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.06967408871929325,\n        \"min\": 0.0,\n        \"max\": 0.381,\n
\"num_unique_values\": 315,\n        \"samples\": [\n          0.21,\n
0.154,\n          0.062\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"liveness\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.11430683210192143,\n        \"min\":
0.0,\n        \"max\": 0.623,\n        \"num_unique_values\": 524,\n
\"samples\": [\n          0.206,\n          0.493,\n          0.242\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"loudness_db\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
2.9762769191750245,\n        \"min\": -18.74,\n        \"max\": 0.0,\n
\"num_unique_values\": 1460,\n        \"samples\": [\n          -
13.62,\n          -10.27,\n          -12.72\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"streams\",\n      \"properties\":
```

{\n   \"dtype\": \"number\",\n   \"std\": 9721026,\n
\"min\": 134,\n   \"max\": 443068342,\n
\"num_unique_values\": 9939,\n   \"samples\": [\n   4517,\
n   8302,\n   3022707\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"viral\",\n   \"properties\": {\
n   \"dtype\": \"number\",\n   \"std\": 0,\n   \"min\":
0,\n   \"max\": 1,\n   \"num_unique_values\": 2,\n
\"samples\": [\n   0,\n   1\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe","variable_name":"df"}

Preprocessing

```
df.shape
```

```
(10000, 17)
```

```
df.columns
```

```
Index(['track_id', 'track_name', 'artist', 'genre', 'release_year',
       'duration_sec', 'danceability', 'energy', 'valence',
'tempo_bpm',
       'acousticness', 'instrumentalness', 'speechiness', 'liveness',
       'loudness_db', 'streams', 'viral'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 17 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   track_id          10000 non-null  object
 1   track_name        10000 non-null  object
 2   artist            10000 non-null  object
 3   genre             10000 non-null  object
 4   release_year      10000 non-null  int64
 5   duration_sec      10000 non-null  int64
 6   danceability      10000 non-null  float64
 7   energy            10000 non-null  float64
 8   valence           10000 non-null  float64
 9   tempo_bpm         10000 non-null  float64
 10  acousticness      10000 non-null  float64
 11  instrumentalness  10000 non-null  float64
 12  speechiness       10000 non-null  float64
 13  liveness          10000 non-null  float64
 14  loudness_db       10000 non-null  float64
 15  streams           10000 non-null  int64
```

```
 16  viral             10000 non-null  int64
dtypes: float64(9), int64(4), object(4)
memory usage: 1.3+ MB
```

```
df.describe()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"release_year\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 3007.536289076137,\n
\"min\": 4.607854476031875,\n        \"max\": 10000.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
2017.4994,\n          2018.0,\n          10000.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"duration_sec\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3467.984421638765,\n        \"min\": 49.96702364665335,\n
\"max\": 10000.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          208.7768,\n          209.0,\n
10000.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"danceability\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 3535.3488940672105,\n        \"min\":
0.0,\n        \"max\": 10000.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          0.6117090000000001,\n          0.624,\n
10000.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"energy\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 3535.340495084876,\n        \"min\": 0.005,\n        \"max\":
10000.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
0.6519382,\n          0.669,\n          10000.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"valence\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
3535.3569272173263,\n        \"min\": 0.0,\n        \"max\": 10000.0,\
n        \"num_unique_values\": 8,\n        \"samples\": [\n
0.5822803000000001,\n          0.582,\n          10000.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"tempo_bpm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3500.9286848376114,\n        \"min\": 19.01720569714834,\n
\"max\": 10000.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          110.97594000000001,\n          111.3,\n
10000.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"acousticness\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 3535.416733354403,\n        \"min\":
0.0,\n        \"max\": 10000.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          0.29558290000000004,\n          0.235,\n
10000.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
```

\"instrumentalness\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 3535.446461317288,\n   \"min\":
0.0,\n   \"max\": 10000.0,\n   \"num_unique_values\": 8,\n
\"samples\": [\n   0.17733740000000003,\n   0.154,\n
10000.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"speechiness\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 3535.494556012271,\n   \"min\":
0.0,\n   \"max\": 10000.0,\n   \"num_unique_values\": 8,\n
\"samples\": [\n   0.087453,\n   0.08,\n
10000.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"liveness\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 3535.4599054153377,\n   \"min\":
0.0,\n   \"max\": 10000.0,\n   \"num_unique_values\": 8,\n
\"samples\": [\n   0.1849249,\n   0.18,\n
10000.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"loudness_db\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 3537.9486102597516,\n   \"min\": -
18.74,\n   \"max\": 10000.0,\n   \"num_unique_values\": 8,\n
\"samples\": [\n   -7.9758960000000005,\n   -7.98,\n
10000.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"streams\",\n   \"properties\": {\n   \"dtype\": \"number\",\
n   \"std\": 155980149.34324956,\n   \"min\": 134.0,\n
\"max\": 443068342.0,\n   \"num_unique_values\": 8,\n
\"samples\": [\n   2415684.8231,\n   329058.0,\n
10000.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"viral\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 3535.3904548547757,\n   \"min\": 0.0,\n   \"max\":
10000.0,\n   \"num_unique_values\": 5,\n   \"samples\": [\n
0.3605,\n   1.0,\n   0.4801695599271473\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe"}

```
df.isna().sum()
```

```
track_id            0
track_name          0
artist              0
genre               0
release_year        0
duration_sec        0
danceability        0
energy              0
valence             0
tempo_bpm           0
acousticness        0
```

```
instrumentalness    0
speechiness         0
liveness            0
loudness_db         0
streams             0
viral               0
dtype: int64
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
numeric_df = df.select_dtypes(include='number')
numeric_df = numeric_df.drop(columns=["release_year"])
numeric_df.head()
```


```
{"summary":"{\n  \"name\": \"numeric_df\",\n  \"rows\": 10000,\n
\"fields\": [\n    {\n      \"column\": \"duration_sec\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
49,\n        \"min\": 60,\n        \"max\": 433,\n
\"num_unique_values\": 299,\n        \"samples\": [\n          92,\n
303,\n          112\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"danceability\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.19261221213990126,\n        \"min\":
0.0,\n        \"max\": 1.0,\n        \"num_unique_values\": 916,\n
\"samples\": [\n          0.173,\n          0.059,\n          0.517\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"energy\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.1876859935743174,\n        \"min\": 0.005,\n        \"max\": 1.0,\n
\"num_unique_values\": 873,\n        \"samples\": [\n          0.122,\
n          0.797,\n          0.357\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"valence\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.1779835378507925,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 873,\n        \"samples\": [\n          0.985,\
n          0.504,\n          0.452\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"tempo_bpm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
19.01720569714834,\n        \"min\": 50.0,\n        \"max\": 177.8,\n
\"num_unique_values\": 974,\n        \"samples\": [\n          114.0,\
n          104.4,\n          122.7\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"acousticness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.2635737017177802,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 980,\n        \"samples\": [\n          0.283,\
```

```
n          0.836,\n          0.302\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"instrumentalness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.16122102897345286,\n        \"min\": 0.0,\n        \"max\": 0.934,\n
\"num_unique_values\": 669,\n        \"samples\": [\n          0.365,\
n          0.024,\n          0.584\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"speechiness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.06967408871929325,\n        \"min\": 0.0,\n        \"max\": 0.381,\n
\"num_unique_values\": 315,\n        \"samples\": [\n          0.21,\n
0.154,\n          0.062\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"liveness\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.11430683210192143,\n        \"min\":
0.0,\n        \"max\": 0.623,\n        \"num_unique_values\": 524,\n
\"samples\": [\n          0.206,\n          0.493,\n          0.242\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"loudness_db\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
2.9762769191750245,\n        \"min\": -18.74,\n        \"max\": 0.0,\n
\"num_unique_values\": 1460,\n        \"samples\": [\n          -
13.62,\n          -10.27,\n          -12.72\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"streams\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 9721026,\n
\"min\": 134,\n        \"max\": 443068342,\n
\"num_unique_values\": 9939,\n        \"samples\": [\n          4517,\
n          8302,\n          3022707\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"viral\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\":
0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0,\n          1\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"numeric_df"}
```


```
numeric_df.mean()
```

```
duration_sec        2.087768e+02
danceability        6.117090e-01
energy              6.519382e-01
valence             5.822803e-01
tempo_bpm           1.109759e+02
acousticness        2.955829e-01
instrumentalness    1.773374e-01
speechiness         8.745300e-02
liveness            1.849249e-01
loudness_db        -7.975896e+00
```

```
streams              2.415685e+06
viral                3.605000e-01
dtype: float64
```

```
numeric_df.median()
```

```
duration_sec           209.000
danceability             0.624
energy                   0.669
valence                  0.582
tempo_bpm              111.300
acousticness             0.235
instrumentalness         0.154
speechiness              0.080
liveness                 0.180
loudness_db             -7.980
streams             329058.000
viral                    0.000
dtype: float64
```

```
numeric_df.mode()
```


```
{"summary":"{\n  \"name\": \"numeric_df\",\n  \"rows\": 1,\n
\"fields\": [\n    {\n      \"column\": \"duration_sec\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 220,\n        \"max\": 220,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          220\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"danceability\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 1.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          1.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"energy\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": null,\n
\"min\": 1.0,\n        \"max\": 1.0,\n        \"num_unique_values\":
1,\n        \"samples\": [\n          1.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"valence\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": null,\n
\"min\": 1.0,\n        \"max\": 1.0,\n        \"num_unique_values\":
1,\n        \"samples\": [\n          1.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"tempo_bpm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 114.5,\n        \"max\": 114.5,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          114.5\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"acousticness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
```

```
null,\n        \"min\": 0.0,\n        \"max\": 0.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"instrumentalness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 0.0,\n        \"max\": 0.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"speechiness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 0.0,\n        \"max\": 0.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"liveness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 0.0,\n        \"max\": 0.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"loudness_db\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 0.0,\n        \"max\": 0.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"streams\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 37774,\n        \"max\": 37774,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          37774\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"viral\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": null,\n
\"min\": 0,\n        \"max\": 0,\n        \"num_unique_values\": 1,\n
\"samples\": [\n          0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe"}
```


Standardize

```
# Feature Engineering
from sklearn.preprocessing import StandardScaler

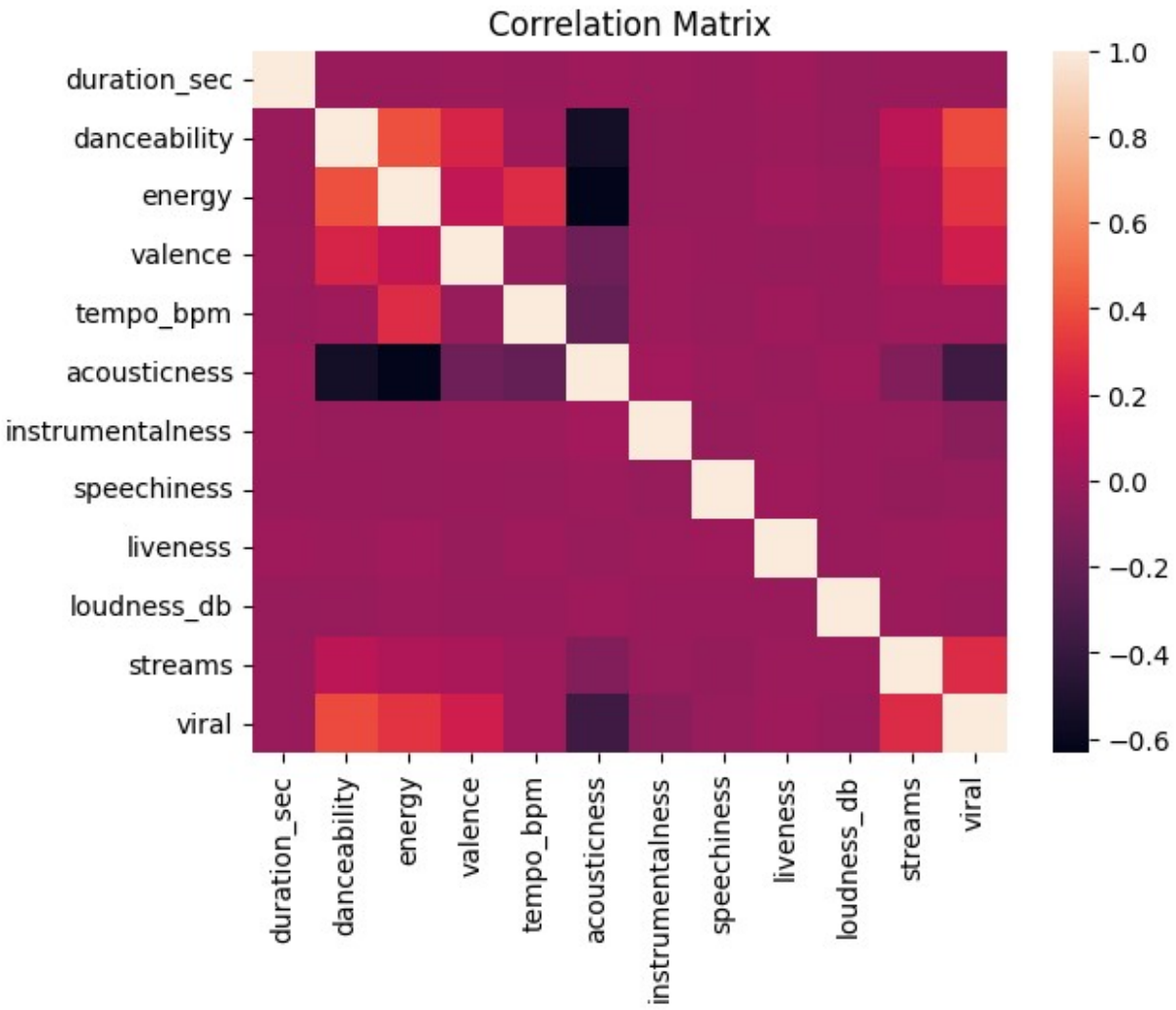
scaler = StandardScaler()
df_scaled = scaler.fit_transform(numeric_df)

df_copy = pd.DataFrame(numeric_df)

# Correlation matrix
corr = df_copy.corr()
sns.heatmap(corr)

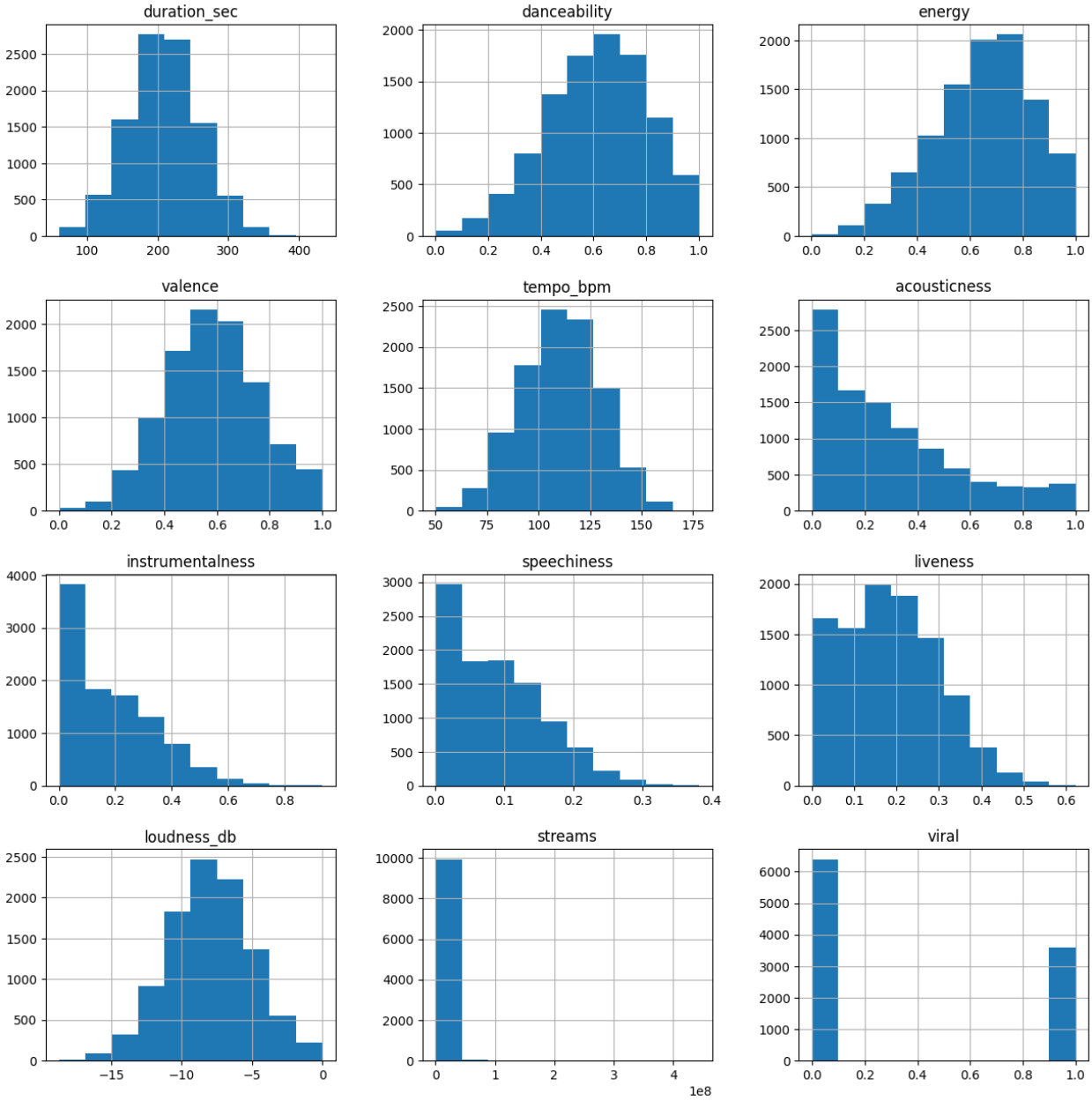
plt.title("Correlation Matrix")
```

```
plt.show()
```

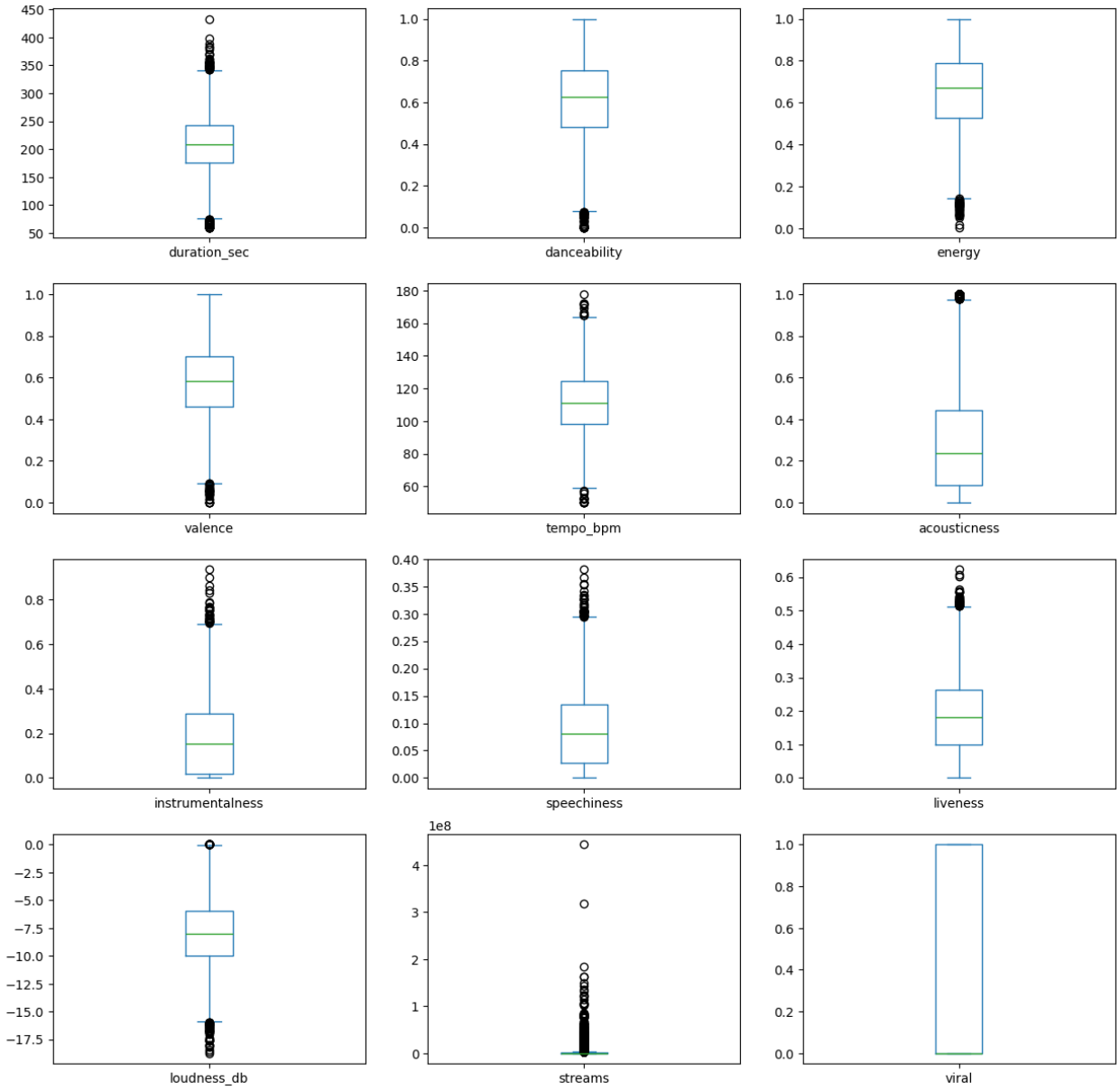


EDA

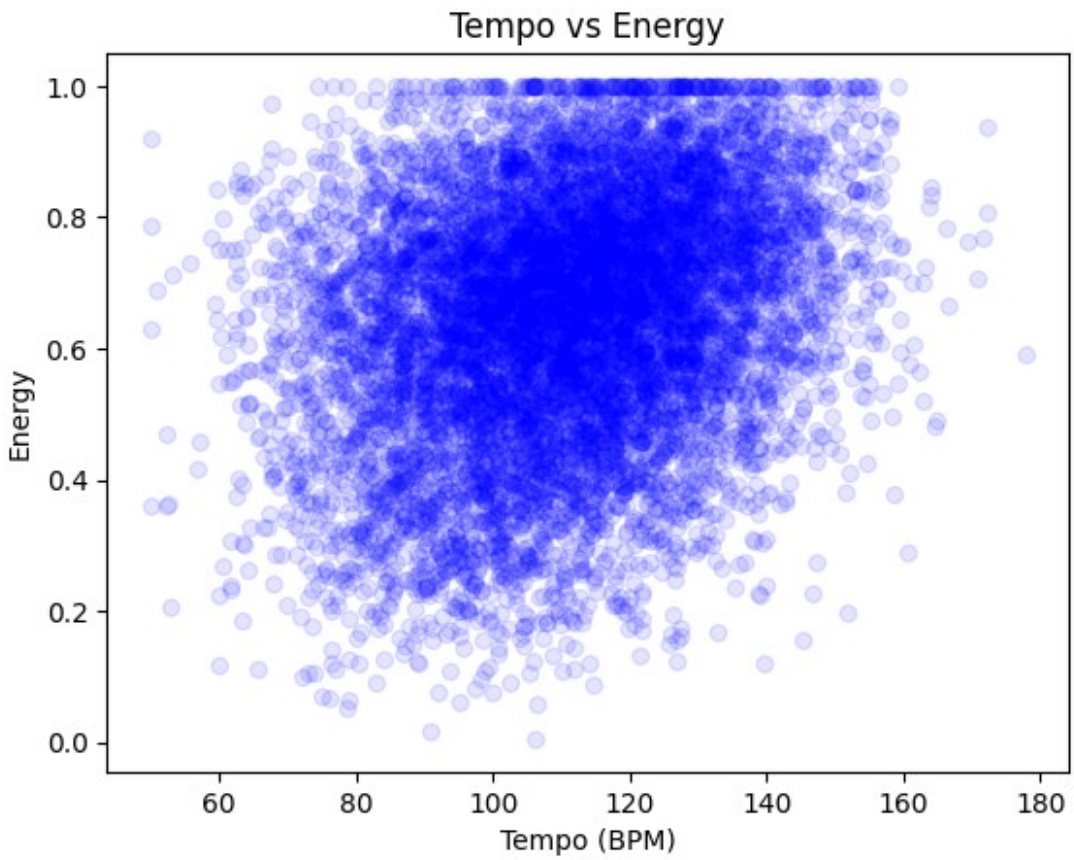
```
# Distribution of each feature
numeric_df.hist(figsize=(15,15))
plt.show()
```



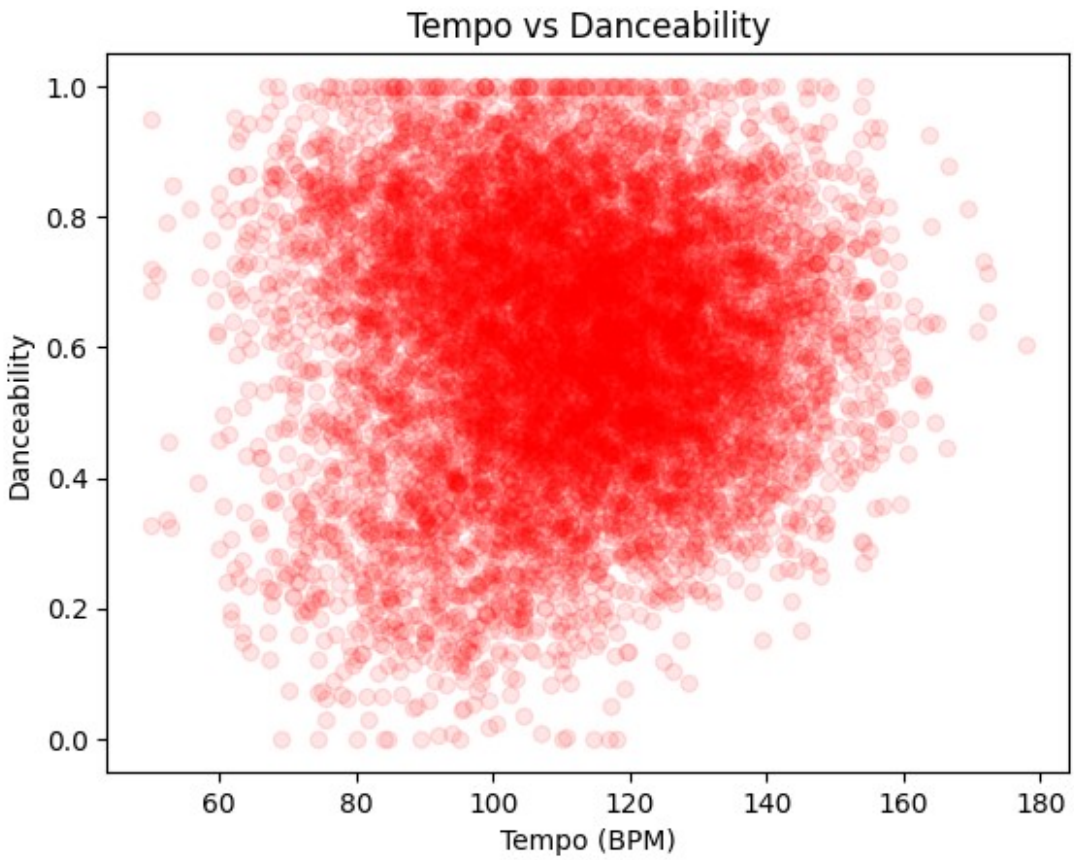

```
# box plot
numeric_df.plot(kind='box', subplots=True, layout=(4,3),
figsize=(15,15))
plt.show()
```




```
# Tempo vs Liveness
plt.scatter(df['tempo_bpm'], df['energy'], alpha=0.1, color="blue")
plt.xlabel('Tempo (BPM)')
plt.ylabel('Energy')
plt.title('Tempo vs Energy')
plt.show()
```




```
# Tempo vs Danceability
plt.scatter(df['tempo_bpm'], df['danceability'], alpha=0.1,
color="red")
plt.xlabel('Tempo (BPM)')
plt.ylabel('Danceability')
plt.title('Tempo vs Danceability')
plt.show()
```



Model Training

- Use k-means clustering to identify the groups of the dataset.
- Use 1 to 10 clusters
- Determine the optimum number of clusters
- Use agglomerative clustering and visualize the dendogram
- Compare the results obtained by the K means and agglomerative clustering.

K-Means testing

```
# KMeans (5 clusters)
kmeans = KMeans(n_clusters=5, init='k-means++', n_init=10,
random_state=42)

# Fit
kmeans.fit(data_scaled)
```

```
KMeans(n_clusters=5, n_init=10, random_state=42)
```

```
# inertia on the fitted data
kmeans.inertia_

# inertia too high
```

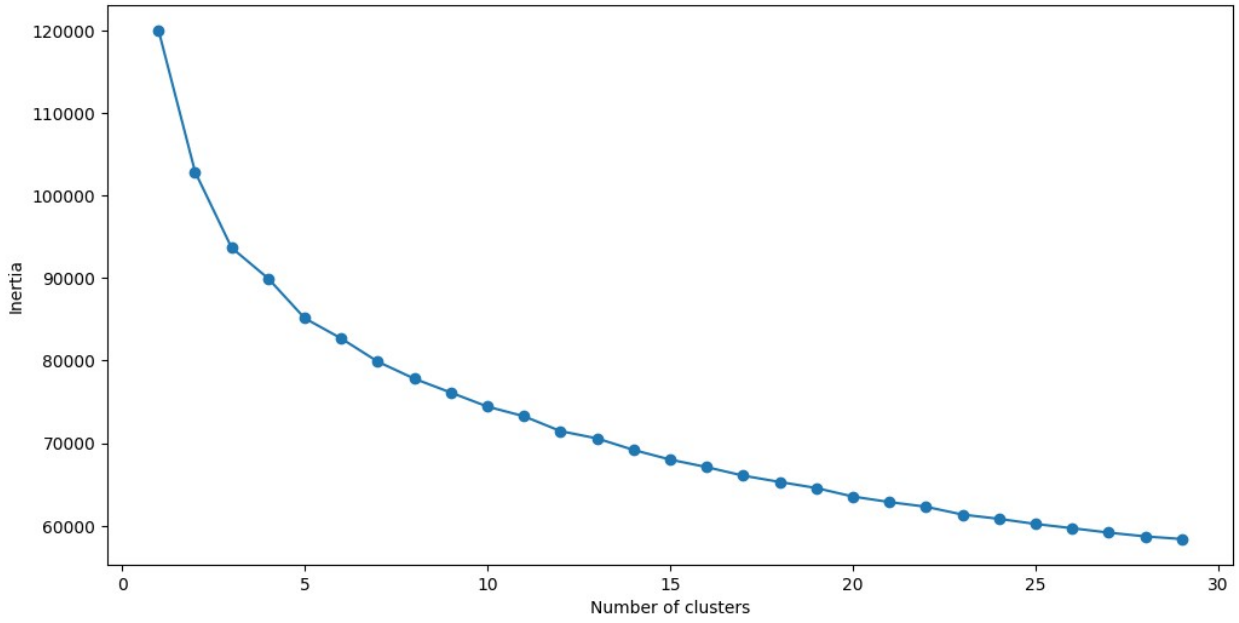
```
85131.0635718932
```

Try multiple cluster values

```
SSE = []
for cluster in range(1,30):
    kmeans = KMeans(n_clusters=cluster, init='k-means++', n_init=10,
random_state=42)
    kmeans.fit(data_scaled)
    SSE.append(kmeans.inertia_)

frame = pd.DataFrame({'Cluster':range(1,30), 'SSE':SSE})
plt.figure(figsize=(12,6))
plt.plot(frame['Cluster'], frame['SSE'], marker='o')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia')
```

```
Text(0, 0.5, 'Inertia')
```



Build a model using 5 clusters and k-means ++ initialization

```
# k means using 5 clusters and k-means++ initialization
kmeans = KMeans(n_clusters=25, init='k-means++')
kmeans.fit(data_scaled)
pred = kmeans.predict(data_scaled)

frame = pd.DataFrame(data_scaled)
frame['cluster'] = pred
frame['cluster'].value_counts()
```

```
cluster
9     555
16    542
5     525
1     518
19    499
20    496
14    487
21    482
3     478
24    471
4     465
2     462
15    439
23    406
6     404
12    404
7     389
0     388
22    376
8     370
11    357
17    339
10    130
13     16
18      2
Name: count, dtype: int64
```

Optimum number: 9

Compute the centroids of the K means cluster

```
centroids = kmeans.cluster_centers_
print(centroids)
```

```
[[-5.96216745e-01 -6.35557582e-01 -8.61003738e-01 -2.46715860e-01
   2.06759533e-01  8.62014048e-01  1.52989804e+00 -4.52336191e-02
   2.97220626e-02 -5.27239665e-02 -1.88794942e-01 -6.91768327e-01]
 [ 9.01860623e-01  7.09051407e-01  2.77658803e-01  2.89148011e-01
  -7.69118162e-01 -5.33113264e-01 -3.97586424e-01 -4.92129742e-01
  -5.58710145e-01  2.70194601e-02  1.29492252e-01  1.32384648e+00]
 [-4.91599073e-01  3.99454780e-01  2.06506971e-02  5.05550184e-01
  -2.48213746e-01 -2.21346925e-01 -1.69519502e-01  1.49356852e+00
  -2.66503928e-01  7.62485216e-03 -1.96632764e-01 -7.50813997e-01]
 [-2.00531833e-01  6.89054078e-01  4.20909429e-01  3.46295598e-01
  -1.60102990e-01 -4.72398115e-01  1.49473169e+00 -1.88860437e-01
  -4.86328688e-02  2.39972911e-02  1.35504123e-01  1.33188780e+00]
 [-5.99445175e-01 -4.94032895e-01 -8.53461494e-01 -3.52247114e-01
   1.62858736e-01  7.93288249e-01 -5.47621186e-01 -4.16295189e-01
```

```
 -8.34676622e-01 -1.48015336e-01 -1.87338642e-01 -6.92587925e-01]
[ 4.30102489e-01  1.85595841e-01  8.80262373e-01 -4.44987506e-01
  1.14628064e+00 -6.46061222e-01 -3.02635881e-01  3.22654351e-02
  4.33482038e-01  9.11093655e-02  1.36327904e-01  1.33188780e+00]
[-1.98894937e-01 -1.63150086e+00 -1.57232801e+00 -7.92362329e-01
 -8.74770088e-01  2.00798853e+00  1.00860373e-03 -6.25788159e-01
  4.80088092e-01  5.26085547e-01 -1.98818674e-01 -7.25037984e-01]
[ 5.02558352e-01  3.73509090e-01  2.61115357e-01  5.27095743e-01
 -2.80917925e-01 -2.66010598e-01  1.26336184e+00 -1.52468103e-02
  9.41886228e-01 -4.80426545e-01 -2.00783435e-01 -7.34752029e-01]
[-1.04799544e-01  3.66553281e-01  5.00902837e-01  6.08897704e-01
  4.36682201e-01 -4.42809959e-01 -2.05315755e-01 -2.49459970e-01
 -5.67283314e-01 -1.68198630e+00  2.01935839e-02  8.59058201e-01]
[-8.56504046e-01  4.33572674e-01  4.51949548e-01  5.35509375e-01
  4.29424304e-01 -4.79439322e-01 -4.71339556e-01 -3.99134685e-01
 -5.70711906e-01  6.75466347e-01  1.35998416e-01  1.32813518e+00]
[-1.35456105e-02  6.48239175e-01  3.06092569e-01  1.92921221e-01
 -6.67737242e-02 -4.23513204e-01 -6.15937281e-02 -2.85728051e-01
 -8.09088940e-02  6.09632567e-02  4.28098880e+00  1.33188780e+00]
[ 5.65369329e-01 -1.57727601e+00 -1.59549344e+00 -6.83461669e-01
 -7.68885267e-01  1.93848187e+00  2.08929891e-01 -2.24414197e-01
 -6.04040880e-01 -7.40236811e-01 -1.99701973e-01 -7.21644504e-01]
[ 8.20469644e-02 -5.69304904e-01 -6.37142873e-01 -3.11636921e-01
  2.43215614e-01  5.86564772e-01 -4.53773866e-01  1.90425068e-02
  1.08174277e+00 -7.15402254e-01 -1.96207141e-01 -6.94106770e-01]
[-3.33272455e-01  7.80316645e-01  5.20505994e-01  4.19131066e-01
  1.31088378e-01 -4.59649263e-01 -1.21887837e-01  2.18664672e-01
  1.59674731e-02  2.52915861e-02  1.30509409e+01  1.33188780e+00]
[ 4.49887324e-02  6.87414339e-01 -1.98351530e-02 -3.80282895e-01
 -1.19132737e+00 -4.63389408e-01 -3.76105525e-01 -4.16026638e-01
 -5.66504353e-02 -1.83961198e-01 -1.89315306e-01 -7.20877831e-01]
[-5.93594364e-02  2.21729004e-01  5.77085958e-01  9.96270574e-02
  4.17969041e-01 -5.75135071e-01  1.41578850e+00 -4.30854882e-01
 -6.97973182e-01  4.88624918e-01 -1.91095958e-01 -7.50813997e-01]
[-1.66872107e-01  6.47517330e-01  2.34431049e-01  6.44926640e-01
 -4.59944389e-01 -4.31592696e-01 -4.53880041e-01 -2.66502511e-01
  1.30442505e+00 -3.64179360e-02  1.36341276e-01  1.30114681e+00]
[-6.11250907e-02 -1.34596858e+00 -1.49669132e+00 -5.41176535e-01
 -8.89309614e-01  1.89084540e+00 -2.76945057e-01  1.38563675e+00
 -1.08948398e-01  2.88479137e-01 -1.97241260e-01 -7.20095681e-01]
[ 2.94673069e-01  4.37642110e-01  6.95674334e-01 -3.21845295e-01
  9.55711846e-01 -5.73237974e-01  1.40598243e+00 -1.04711306e+00
  2.67496074e-01  3.76628949e-01  3.88764103e+01  1.33188780e+00]
[ 3.68348235e-02  1.24923243e-01  1.99751338e-01  1.29176869e+00
  7.08186393e-02 -1.67741944e-01 -4.42267920e-01 -4.98654737e-01
 -3.05551696e-01  3.91257713e-01 -2.00243479e-01 -7.46640246e-01]
[ 4.34046147e-01 -1.38349652e-01  7.55789753e-01 -7.42313413e-01
  9.48640222e-01 -5.52331654e-01 -4.06361771e-01 -6.57578431e-01
 -2.82374376e-01 -3.40271882e-01 -2.09914490e-01 -7.50813997e-01]
```

```
[-5.81498712e-02  6.89078957e-01  2.65363191e-01  2.24936420e-01
 -3.29473871e-01 -5.18458300e-01 -2.76902691e-01  1.50883847e+00
 -2.25124257e-01 -7.95459223e-02  1.16544897e-01  1.33188780e+00]
[ 4.79218897e-01 -3.20823565e-01  7.47030216e-01 -7.23259170e-01
  8.37923479e-01 -5.24892261e-01 -5.38118371e-02  1.33682527e+00
  8.87967536e-02 -2.45296851e-01 -1.97560072e-01 -7.17579394e-01]
[ 1.27694634e+00 -4.25003563e-01 -5.49760126e-01 -5.56868212e-02
  1.92999050e-01  4.61353008e-01 -1.58817310e-01  1.06450319e-01
 -8.04527220e-02  9.46847796e-01 -1.98737103e-01 -6.99515923e-01]
[-1.01561963e+00 -6.10250699e-02  5.64112479e-01 -5.08076176e-01
  5.99836842e-01 -4.55460846e-01 -2.26651113e-01 -2.15675584e-01
  8.57075587e-01  6.26711643e-01 -2.06750427e-01 -7.28704636e-01]]
```

Fit Agglomerative clustering model

```
from sklearn.cluster import AgglomerativeClustering

ag = AgglomerativeClustering(n_clusters=25, linkage='ward',
compute_full_tree=True)
ag = ag.fit(data_scaled)
df['agglom'] = ag.fit_predict(data_scaled)
```

Visualize dendrogram

```
from scipy.cluster import hierarchy
from matplotlib import colors
import seaborn as sns

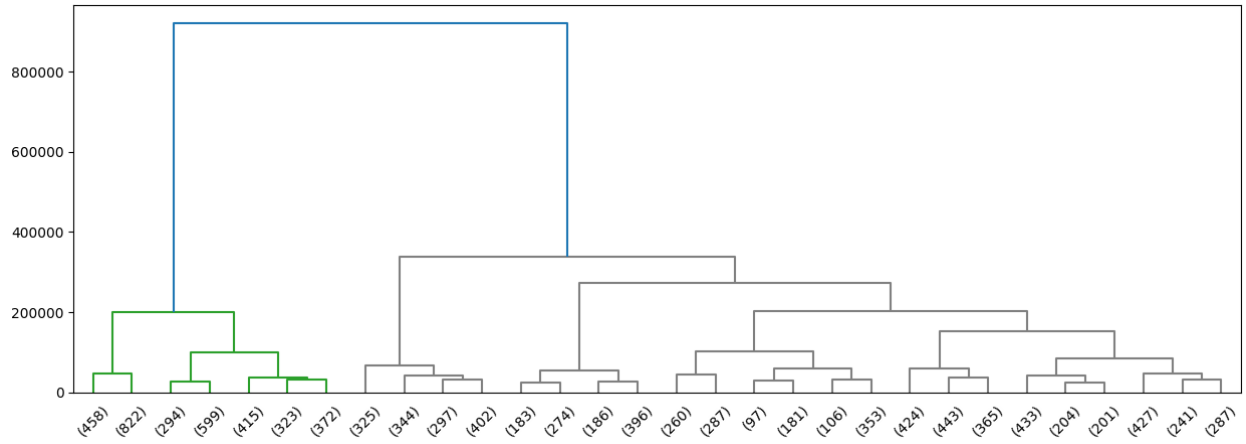
Z = hierarchy.linkage(ag.children_, method='ward')

fig, ax = plt.subplots(figsize=(15,5))

dark_palette = sns.color_palette()
red = colors.to_hex(dark_palette[2])
blue = colors.to_hex(dark_palette[0])

hierarchy.set_link_color_palette([red, 'gray'])

den = hierarchy.dendrogram(Z, orientation='top',
                           p=30, truncate_mode='lastp',
                           show_leaf_counts=True, ax=ax,
                           above_threshold_color=blue)
```




```
# Compare performances of models using silhouette scores
from sklearn.metrics import silhouette_score

# Calculate silhouette scores (using a sample of 5000 if dataset is
large, or full dataset)
print("Calculating silhouette scores (this might take a few
moments)... ")

# K-Means Silhouette Score
kmeans_silhouette = silhouette_score(data_scaled, pred,
metric='euclidean', random_state=42)

# Agglomerative Silhouette Score
ag_labels = df['agglom']
agglom_silhouette = silhouette_score(data_scaled, ag_labels,
metric='euclidean')

print(f"\nK-Means Clustering Silhouette Score (25 clusters):
{kmeans_silhouette:.4f}")
print(f"Agglomerative Hierarchical Clustering Silhouette Score (25
clusters): {agglom_silhouette:.4f}")

# Summary analysis
if kmeans_silhouette > agglom_silhouette:
    print("\nSummary: K-Means has a higher Silhouette Score,
indicating more compact and well-separated clusters on average.")
elif agglom_silhouette > kmeans_silhouette:
    print("\nSummary: Agglomerative Clustering has a higher Silhouette
Score, indicating better-defined cluster boundaries for this
hierarchical layout.")
else:
    print("\nSummary: Both models performed equally in terms of
silhouette-based compactness and separation.")
```

```
Calculating silhouette scores (this might take a few moments)...
```

```
K-Means Clustering Silhouette Score (25 clusters): 0.0760
Agglomerative Hierarchical Clustering Silhouette Score (25 clusters): 0.0202

Summary: K-Means has a higher Silhouette Score, indicating more compact and well-separated clusters on average.
```

Conclusion

The activity mainly tackles on 2 clustering unsupervised learning models, namely K-Means and Agglomeration Clustering. In accomplishing the activity, we analyzed the Wholesale Customers Dataset in the procedure part. Before applying these models, a series of preprocessing, cleaning, and exploratory data analysis (EDA) is needed to explore the data more and know the highlights of the data points. After this, we built K-Means clustering models and mapped the Sum of Squared Errors (SSE) over several cluster counts using the Elbow Method to identify the optimal cluster groupings. We then constructed an Agglomerative Hierarchical Clustering model and visualized the dataset's structural groupings using a dendrogram. After model training, we compared their results and defined their performances. These procedure were also made in the supplementary activity, which tackled on What Makes a Song Go Viral dataset, and it showed that the K Means clustering model performed better in it. In conclusion, the use of these clustering models are based on the applied dataset, since each model provides different performances and applications